

OCaml 版 AIPL 機能一覧ユーザガイド

児玉靖司 (Yasushi Kodama)

法政大学 (Hosei University)

2026 年 5 月 12 日

目次

1	概要	2
2	対象実装	2
3	言語構文	2
4	データ型と式	3
5	アクターと並行実行	3
6	組み込み関数	4
6.1	基本 I/O	4
6.2	数学関数	4
6.3	配列	4
6.4	SDL 描画	4
6.5	Web/リモート連携	5
6.6	AI 呼び出し	5
7	REPL/バッチ実行	5
8	型推論/型関連	6
9	トランスレータ/派生実行系	6
10	サンプル	7
10.1	基本	7
10.2	並行/同期	7
10.3	描画	7
10.4	AI/リモート	7
11	代表的な実行コマンド	8
12	現時点の主な制限	8

1 概要

OCaml 版 AIPL は、Actor-based Intelligent Parallel Language の OCaml 実装である。各 `class` のインスタンスはアクターとして生成され、メッセージキューと OCaml Thread によって並行実行される。

Python 版 AIPL にある Phase 11 以降の新しい表面構文、たとえば引数型注釈、`pub`、`linear T`、能力エフェクト注釈、`transient cast` 構文などは、OCaml 版のパーサでは直接扱わない。ただし、OCaml 版にも HM 風の簡易型推論、`now`、`future`、`await`、リモートアクター、AI 呼び出し、SDL 描画、Web gateway などの実行機能がある。

2 対象実装

項目	ファイル
ランタイム本体	src/eval_thread.ml
構文定義	src/parser.mly, src/lexer.mll
AST	src/ast.ml
型推論プレリユード	src/typing_env.ml, src/infer.ml
REPL/バッチ実行	src/repl_thread.ml
サンプル	abclc/*.abcl, abclc/ai-samples/*.abcl

3 言語構文

分類	機能	例
クラス定義	<code>class Name { ... }</code>	<code>class Hello { ... }</code>
フィールド	<code>var x = expr;、float x = expr;</code>	<code>var count = 0;</code>
メソッド	<code>method name(params) { ... }</code>	<code>method greet() { print("hi"); }</code>
オブジェクト生成	<code>new Class(args)</code>	<code>var h = new Hello(5);</code>
自動初期化	<code>init</code> メソッドがあれば生成時に送信	<code>new Hello(5)</code> で <code>init(5)</code>
代入	<code>x = expr;</code>	<code>count = count + 1;</code>
ローカル変数	<code>var x = expr;</code>	<code>var i = 0;</code>
関数呼び出し	<code>f(args)</code> または <code>call f(args);</code>	<code>print(x);</code>
非同期送信	<code>send target.method(args);</code>	<code>send h.greet();</code>
unsafe 送信	<code>send! target.method(args);</code>	<code>send! h.greet();</code>
同期呼び出し	<code>now target.method(args)</code>	<code>var r = now calc.add(1, 2);</code>
Future	<code>future target.method(args)</code>	<code>var f = future w.ask(q);</code>
Await	<code>await expr</code>	<code>var r = await f;</code>
リモート対象	<code>remote("host:port", "actor")</code>	<code>now remote("localhost:9100","reviewer").review</code>

条件分岐	<code>if (cond) stmt else stmt</code>	<code>if (n < 10) { ... }</code>
ループ	<code>while cond do stmt</code>	<code>while i < 10 do { ... }</code>
ブロック	<code>{ stmt ... }</code>	<code>{ x = x + 1; print(x); }</code>
振る舞い変更	<code>become Class(args);</code>	<code>become B();</code>
メールボックス選択	<code>select { case m(args) -> { ... } timeout ms -> { ... } }</code>	<code>select { case put(x) -> { ... } }</code>
特殊変数	<code>self, sender</code>	<code>send self.tick();</code>
コメント	<code>// ..., /* ... */</code>	<code>// line comment</code>

4 データ型と式

OCaml 版 AIPL は実行時値として、整数、浮動小数、文字列、真偽値、Unit、Actor 参照、Array、Future を扱う。算術演算は `+`, `-`, `*`, `/`、比較演算は `>`, `<`, `>=`, `<=`, `==`, `!=` を使う。`+` は片側が文字列の場合、文字列連結として扱われる。

分類	内容
整数	123
浮動小数	123., 123.45
文字列	"hello"
真偽値	比較式などから生成される内部値
Unit	組み込み関数や文の戻り値
Actor 参照	<code>new</code> で生成されたアクター
Array	<code>array_empty</code> , <code>array_push</code> , <code>array_get</code> , <code>array_set</code> で操作
Future	<code>future</code> の戻り値

5 アクターと並行実行

- `new Class(args)` でインスタンスごとにアクターを作成する。
- 各アクターは自身のメッセージキューを持つ。
- 各アクターは OCaml Thread でループ実行される。
- `send` は送信後に待たない。
- `now` は受信側の `reply(value)` まで待つ。
- `future` は即座に Future ハンドルを返す。
- `await` は Future の完了まで待つ。
- `now` と `future` は内部の `msg_id` と `reply slot` で関連される。
- `sender` はメッセージ送信元のアクター名を保持する。
- `self` は現在のアクター自身を表す。
- `become` はアクターのクラス/メソッド表を差し替え、既存状態を保持する。

6 組み込み関数

6.1 基本 I/O

関数	概要
<code>print(x)</code>	値を標準出力と Web ログに出力する。
<code>typeof(x)</code>	実行時の型名を文字列で返す。
<code>reply(x)</code>	<code>now</code> / <code>future</code> の呼び出し元へ値を返す。
<code>wait(ms)</code>	ミリ秒単位でスリープする。

6.2 数学関数

数学関数として、`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sqrt`, `exp`, `log10`, `abs`, `floor`, `ceil`, `round` が利用できる。

6.3 配列

関数	概要
<code>array_empty()</code>	空配列を作る。
<code>array_len(xs)</code>	配列長を返す。
<code>array_get(xs, i)</code>	要素を読む。
<code>array_set(xs, i, v)</code>	要素を更新した新しい配列を返す。
<code>array_push(xs, v)</code>	末尾に追加した新しい配列を返す。

6.4 SDL 描画

関数	概要
<code>sdl_init(w, h)</code>	SDL ウィンドウを初期化する。
<code>sdl_clear()</code>	画面をクリアする。
<code>sdl_present()</code>	描画結果を表示する。
<code>sdl_line(x1, y1, x2, y2)</code>	線を描く。
<code>sdl_erase_line(x1, y1, x2, y2)</code>	線を背景色で消す。
<code>sdl_line_c(x1, y1, x2, y2, r, g, b)</code>	色付き線を描く。
<code>sdl_poll_key()</code>	キー入力を非ブロッキング取得する。
<code>sdl_mouse_x()</code>	マウス X 座標を返す。

<code>sdl_mouse_y()</code>	マウス Y 座標を返す。
<code>sdl_mouse_down()</code>	左ボタン押下状態を返す。

SDL_VIDEODRIVER=dummy で実行するとウィンドウは出ない。これは CI やヘッドレス確認用の設定である。

6.5 Web/リモート連携

関数/構文	概要
<code>web_listen(port)</code>	HTTP gateway を起動する。
<code>web_expose(path, actor)</code>	actor を Web endpoint として公開する。
<code>send remote("host:port", "actor").method(args);</code>	HTTP 越しの非同期送信。
<code>now remote("host:port", "actor").method(args)</code>	HTTP 越しの同期呼び出し。
<code>future remote("host:port", "actor").method(args)</code>	HTTP 越しの Future 呼び出し。

主な HTTP API は `src/web_gateway.ml` にあり、JSON 形式の `/api/json/send` と `/api/json/call` を使う。

6.6 AI 呼び出し

関数	概要
<code>ai_call(prompt)</code>	LLM へプロンプトを送る。
<code>ai_call_with_system(system, prompt)</code>	system prompt 付きで LLM 呼び出しを行う。
<code>ai_call_retry(max_attempts, prompt)</code>	リトライ付き AI 呼び出しを行う。
<code>ai_call_retry_with_system(max_attempts, system, prompt)</code>	system prompt 付きリトライを行う。
<code>ai_usage()</code>	使用量文字列を返す。
<code>ai_remaining()</code>	残りトークン予算を返す。未設定なら -1。
<code>ai_cost()</code>	推定コストを返す。

関連する環境変数は、`ABCL_AI_PROVIDER`, `GEMINI_API_KEY`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `ABCL_AI_TOKEN_BUDGET`, `ABCL_AI_MAX_CONCURRENT`, `ABCL_AI_USAGE_FILE` である。

7 REPL/バッチ実行

OCaml 版の REPL 実行体は `src/abclrepl_thread` または Dune ビルド後の `_build/default/src/repl_thread.exe` である。

コマンド	概要
help	コマンド一覧を表示する。
load file.abcl	.abcl ファイルを読み込み、AST 表示と型チェックを行う。
compile	ロード済みプログラムを登録/起動する。
send obj.method(args)	REPL から非同期メッセージを送信する。
ssend obj.method	引数なし簡易送信を行う。
ast name	クラスまたはインスタンスの AST を表示する。
pprint name	クラス定義を整形表示する。
script file.bat	REPL コマンドをまとめて実行する。
clear	表示/バッファ系をクリアする。
reset	セッション状態をリセットする。
exit / quit	終了する。

バッチ実行例を示す。

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-clojure/abcl
eval $(opam env --switch=ocaml-5.1.1 --set-switch)
ocamlrun ./src/abclrepl_thread -f com04.bat
```

com04.bat のような .bat ファイルは、通常次の形式である。

```
load LDD.abcl
compile
```

8 型推論/型関連

- HM 風の型推論は src/infer.ml と src/typing_env.ml で実装される。
- 算術型は int と float を扱い、混在時は float へ昇格する。
- 比較型は数値比較と文字列の == / != を扱う。
- print は多相的に任意値を表示できる扱いである。
- reply は多相的に任意値を返せる扱いである。
- array.* には多相型が付与される。
- actor_dump と typeof でアクター型を確認できる。

9 トランスレータ/派生実行系

機能	ファイル	概要
C 変換	src/c_translator.ml	AIPL AST から C ランタイム風コードを生成する。
Xinu 向け変換	src/c_translator.ml	Xinu 向けの出力パスを含む。
Python 変換	src/c_translator.ml	Python アクターコード生成パスを含む。

Web gateway	src/web_gateway.ml	ブラウザ/HTTP からアクターへ送信する。
Browser 連携	src/browser-abcl/	ブラウザ側 AIPL 実装/デモ。
GUI IDE	src/gui_ide.ml	SDL ベースの IDE。
TUI IDE	src/tui_ide.ml	ターミナル UI。

10 サンプル

10.1 基本

- abclc/Hello.abcl: 生成、init、send、フィールド更新。
- abclc/counter.abcl: カウンタ。
- abclc/PingPong.abcl: 2 アクター間メッセージ。
- abclc/become.abcl: become による振る舞い変更。
- abclc/bbecome.abcl: 状態を持つ become。

10.2 並行/同期

- abclc/Phase11_TypedCounter.abcl: now と reply を使う計算。
- abclc/Phase13_Channels.abcl: チャンネル相当の同期サンプル。
- abclc/bounded_buffer.abcl: bounded buffer。
- abclc/Philosophers5.abcl: 食事する哲学者。

10.3 描画

- abclc/LineDrawer.abcl: 線描画。
- abclc/Rotate4Lines.abcl: 4 本線の回転。
- abclc/Rotate4LinesGui.abcl: GUI 操作付き回転。
- abclc/Philosophers5Gui.abcl: 哲学者 GUI。
- abclc/bounded_buffer_visual.abcl: bounded buffer 可視化。

10.4 AI/リモート

- abclc/ai-samples/AIHello.abcl: LLM 呼び出し。
- abclc/ai-samples/Budgeted.abcl: AI 予算/同時実行管理。
- abclc/ai-samples/CooperativeNowFuture.abcl: 複数 AI 役割の協調。
- abclc/ai-samples/CooperativeNowFuture-jp.abcl: 日本語版 AI 協調。
- abclc/ai-samples/CooperativeNowFuture-jp-remote.abcl: リモート AI actor 連携。
- abclc/ai-samples/RemoteCalcServer.abcl: リモート計算サーバ。
- abclc/ai-samples/RemoteCalcClient.abcl: リモート計算クライアント。

11 代表的な実行コマンド

通常実行:

```
cd /Users/yaskodama/local-genai-chatgpt-ga/aio-cs-ai/abclc
eval $(opam env --switch=ocaml-5.1.1 --set-switch)
ocamlrun ../src/abclrepl_thread -f run_pingpong.bat
```

描画なしのヘッドレス確認:

```
SDL_VIDEODRIVER=dummy SDL_RENDER_DRIVER=software \
ocamlrun ../src/abclrepl_thread -f com04.bat
```

AI を mock で実行:

```
ABCL_AI_PROVIDER=mock \
ocamlrun ../src/abclrepl_thread -f ai-samples/run_aihelloworld.bat
```

12 現時点の主な制限

- `SDL_VIDEODRIVER=dummy` では実画面描画は表示されない。
- OCaml 版は Python 版より古い表面構文を使うため、型注釈付きメソッドや所有権構文はそのままでは読めない。
- `select` は AST/構文上に存在するが、サンプルと実装状況に依存するため、実運用前に対象サンプルで確認すること。
- リモート/AI 機能は環境変数、API key、ネットワーク、`curl/jq` など外部コマンドに依存する。
- ビルド済み `src/abclrepl_thread` は OCaml runtime のバージョンに依存するため、magic mismatch が出る場合は同じ `opam switch` で再ビルドする。